



Singleton no Flask com SQLAlchemy



Flask

+

SQLAlchemy

Como usar nos casos de uso com MySQL

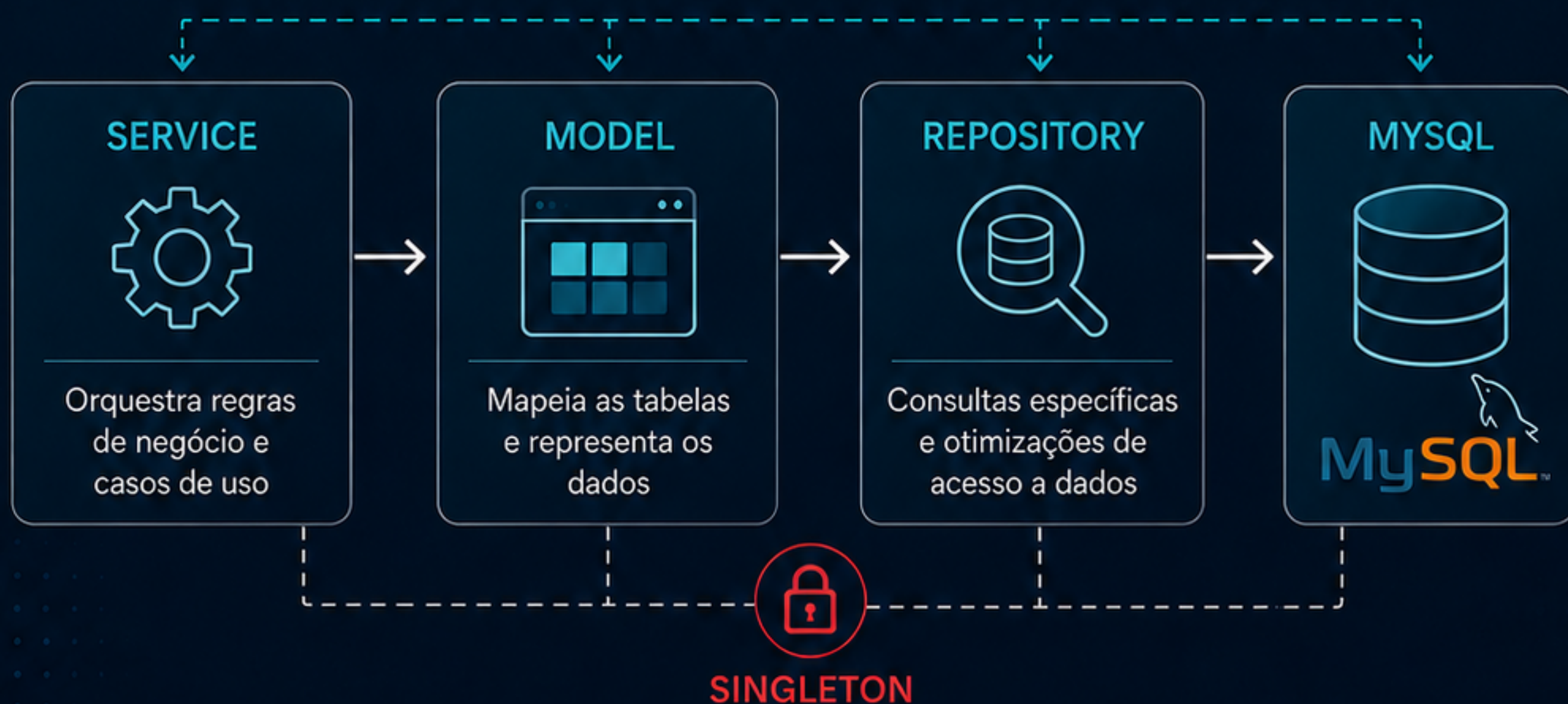
Quando o Service usa o Model?

Quando usa o Repository?

E onde entra o Singleton?



CRUD no Model • Consultas específicas no Repository



Singleton: definição e nosso contexto

Singleton é um padrão de projeto que garante uma **única instância** e um **ponto global de acesso** a ela.

No nosso contexto, essa instância única é o objeto **db** do SQLAlchemy.



1 É criado **uma vez**



2 É **compartilhado** pela aplicação



3 **Centraliza** o acesso ao banco

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
```

SERVICE



usa o db

MODEL



usa o db

REPOSITORY



usa o db



SQLAlchemy

db (instância única)



No nosso projeto, **Singleton** = o mesmo **db** reutilizado por **Models** e **Repositories**.



CRUD básico fica no **Model**

Quando a operação é simples, o caso de uso pode usar o **Model** diretamente.



Flask

+

SQLAlchemy

Exemplos de CRUD

- salvar()
- atualizar()
- remover()
- buscar_por_id(id)
- listar_todos()

```
class Usuario(db.Model):  
    def salvar(self):  
        db.session.add(self)  
        db.session.commit()
```



Se é **CRUD simples**, o Service pode chamar o **Model**.



CriarUsuarioService



Usuario.salvar()



MySQL





Repository é para o que **não é** CRUD

Use **Repository** quando a operação extrapola o CRUD simples.



Flask



SQLAlchemy



consultas
com join



filtros
específicos



relatórios



procedures
complexas



rankings e
estatísticas



```
db.session.execute(  
    text("CALL sp_buscar_usuarios_ativos_com_perfil()")  
)
```



Repository **não substitui** o Model. Ele **complementa**.



ListarUsuarios
AtivosService



Usuario
Repository



Procedure /
Consulta
específica



Como o Service decide?



Flask

+

SQLAlchemy

O Service representa o caso de uso e decide qual caminho seguir.



Se for CRUD



Use o Model

```
CriarUsuarioService ->  
Usuario.salvar()
```



Ex.: criar, atualizar, remover,
buscar por ID



Se não for CRUD



Use o Repository

```
ListarUsuariosAtivosService ->  
UsuarioRepository.buscar_  
usuarios_ativos_com_perfil()
```



Ex.: joins, filtros, relatórios,
procedures

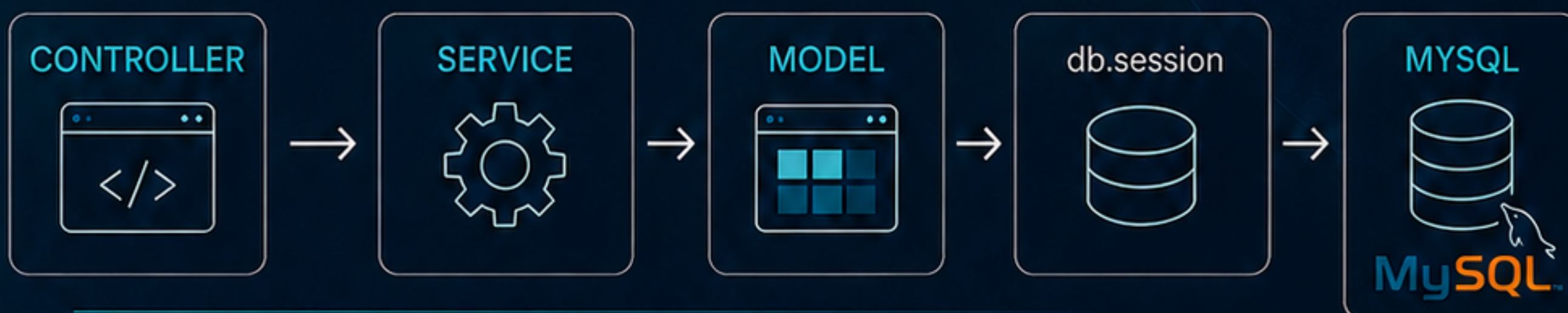


Service escolhe a **estratégia**; Singleton sustenta a **infraestrutura**.

Fluxo dos casos de uso

Veja como o Singleton aparece indiretamente em cada fluxo.

Fluxo 1 • CRUD



 Exemplo: criar usuário

Fluxo 2 • Operação específica



 Exemplo: listar usuários ativos com perfil



Em ambos os fluxos, o acesso ao banco passa pelo mesmo **db** compartilhado.



Resumo **prático**

1



Singleton =
instância única do SQLAlchemy

2



Model =
operações CRUD

3



Repository =
operações que extrapolam o CRUD

4



Service =
caso de uso que decide
qual caminho seguir



Regra de bolso: CRUD no Model.
Consulta específica no Repository.



Salve este carrossel para revisar depois.

